

INGENIERÍA EN SISTEMAS COMPUTACIONALES

Proyecto Final Lenguaje ensamblador

Profesor: Irving Cardona Camacho

4-B

Alumnos:

Vladimir Suarez Vega
Paola Sarahi Zermeño Ojeda
Diana Valeria Zermeño Ojeda

Descripción general del juego.

BitQuest: Explorador de Matrices con C y NASM es un videojuego de aventura en consola desarrollado mediante programación híbrida en lenguaje C y NASM de 64 bits. El jugador deberá explorar extensos laberintos representados por matrices de gran tamaño, donde tendrá que recolectar monedas, encontrar una llave especial, abrir una puerta bloqueada y localizar la salida para completar cada nivel.

A lo largo de tres niveles progresivos, el usuario recorrerá mapas de 60×60 posiciones mientras observa únicamente una ventana visible de 20×20 celdas, lo que genera una experiencia de exploración dinámica donde la vista se desplaza conforme avanza el personaje. Esta mecánica obliga al jugador a descubrir el entorno poco a poco y planificar cuidadosamente sus movimientos.

El juego incorpora distintos elementos interactivos como paredes, caminos libres, monedas, llaves, puertas y salidas, cada uno con una función específica dentro de la aventura. Además, registra estadísticas como monedas recolectadas, pasos realizados y puntaje obtenido, proporcionando un resumen al finalizar cada nivel y al completar el juego.

Desde el punto de vista técnico, BitQuest demuestra la integración entre C y NASM mediante la implementación de funciones en ensamblador encargadas de realizar tareas como el conteo de monedas, validación de movimientos, detección de objetos, cálculo de puntajes y análisis de celdas del mapa. De esta manera, el proyecto combina conceptos de estructuras de datos, manejo de memoria, matrices bidimensionales, lógica de videojuegos y programación de bajo nivel para ofrecer una experiencia interactiva y educativa.

En conjunto, BitQuest no solo representa un desafío de exploración para el jugador, sino también una demostración práctica de cómo la programación en C puede complementarse con rutinas escritas en lenguaje ensamblador para desarrollar aplicaciones más cercanas al funcionamiento interno del hardware.

Explicación de las funciones

main.c

Función	Propósito	Parámetro de entrada	Valor de retorno	Descripción
contarCaracter	Contar cuántas veces aparece un carácter en el mapa.	rdi: mapa, esi: total de celdas, edx: carácter a buscar.	Número de coincidencias en eax.	Recorre toda la matriz y cuenta cuántas veces aparece el carácter indicado. Se utiliza principalmente para obtener el total de monedas (M) de cada nivel.
validarMovimiento	Verificar si una posición es válida para moverse.	rdi: mapa, esi: columnas, edx: fila destino, ecx: columna destino.	1 si el movimiento es válido, 0 si está bloqueado.	Calcula la posición dentro de la matriz y comprueba si contiene una pared (#). Si no es una pared, permite el movimiento.
calcularPuntaje	Calcular la puntuación final del jugador.	edi: monedas, esi: llaves, edx: pasos, ecx: niveles completados.	Puntaje total en eax.	Aplica la fórmula: $(\text{Monedas} \times 100) + (\text{Llaves} \times 500) + (\text{Niveles} \times 1000) - (\text{Pasos} \times 2)$.
detectarObjeto	Detectar si existe un objeto específico en una posición.	rdi: mapa, esi: columnas, edx: fila, ecx: columna, r8: carácter a buscar.	1 si el objeto existe, 0 si no existe.	Permite verificar si una celda contiene una moneda (M), llave (K), puerta (D), salida (E) u otro elemento.
contarCeldasLibres	Contar la cantidad de caminos disponibles.	rdi: mapa, esi: total de celdas.	Total de celdas libres en eax.	Recorre toda la matriz y cuenta las posiciones que contienen el carácter '.' (camino libre).
actualizarMapa	Actualizar la posición del jugador después de un movimiento.	rdi: mapa, esi: columnas, edx: fila actual, ecx: columna actual, r8: fila destino, r9: columna destino.	No devuelve valor	Mueve el personaje a la nueva posición copiando el carácter del jugador a la celda destino y reemplazando la posición anterior por '.'.

juego.c

Función	Propósito	Parámetros	Valor de retorno	Descripción
iniciarJuego()	Inicializar juego	Ninguno	No devuelve valor	Carga el primer nivel mediante cambiarNivel() e inicializa los gráficos y recursos necesarios para comenzar la partida.
estadoJuego()	Controlar toda la lógica del juego	Ninguno	No devuelve valor	Gestiona el movimiento del jugador, la detección de objetos, las colisiones, el cambio de nivel, las estadísticas y la actualización de la cámara
*llamarDibujar(bool status)	Dibujar todos los elementos en pantalla	"status" controla si el juego continúa ejecutándose.	No devuelve valor	Muestra el mapa, jugador, panel de información, pantalla de fin de nivel y pantalla final del juego según el estado actual.
limpiar()	Liberar recursos	Ninguno	No devuelve valor	Cierra los recursos gráficos y de audio al terminar el juego
cambiarNivel()	Cargar nuevo nivel	Ninguno	No devuelve valor	Reinicia variables del nivel actual, libera el mapa anterior y carga el mapa correspondiente al siguiente nivel.
*cargarMapa(const char nombreArchivo, int mapSize)	Leer mapa desde un archivo de texto	nombreArchivo: nombre del archivo, mapSize: tamaño del mapa.	Puntero al mapa cargado (char)	Abre el archivo, reserva memoria y almacena todos los caracteres del mapa en una matriz lineal.

“Variables globales”

Variable	Propósito
camX, camY	Posición de la cámara dentro del mapa
jugadorX, jugadorY	Coordenadas actuales del jugador.
Nivel	Nivel actual del juego.
mapSize	tamaño del mapa cargado.
mapActual	Puntero al mapa que se encuentra cargado en memoria.
Estado	Estado actual del juego (1=Jugando, 2=Fin de nivel, 3=Fin del juego).
monedasObtNivel	Monedas obtenidas en el nivel actual.
llavesObtNivel	Llaves obtenidas en el nivel actual.
pasosNivel	Pasos realizados en el nivel actual.
monedasAcum	Monedas acumuladas durante toda la partida.
llavesAcum	Llaves acumuladas en toda la partida
pasosAcum	Pasos acumulados durante toda la partida.
nivelesComp	Número de niveles completados.
puntajeFinal	Puntaje final calculado al terminar el juego.

“Estados del juego”

Estado	Valor	Descripción
Jugando	1	El jugador está explorando el mapa y puede moverse.
Fin del nivel	2	Se muestran estadísticas del nivel completado.
Fin del juego	3	Se muestran estadísticas finales y el puntaje total.

“Acciones del jugador”

Tecla	Acción
W	Mover hacia arriba.
A	Mover hacia la izquierda.
S	Mover hacia abajo.
D	Mover hacia la derecha.
Q	Salir del juego.
ENTER	Continuar al siguiente nivel o salir al finalizar el juego.

“Objetos detectados en el mapa”

Símbolo	Descripción	Acción al Interactuar
P	Jugador	Representa la posición actual del personaje.
.	Paso Libre	Permite el movimiento.
#	Pared	Bloquea el movimiento.
M	Moneda	Incrementa el contador de monedas.
K	Llave	Permite abrir puertas.
D	Puerta	Consume una llave para atravesarla.
E	Salida	Completa el nivel actual.

“Funciones NASM utilizadas”

Función NASM	Uso dentro del Código C
contarCaracter()	Cuenta monedas y llaves existentes en el mapa.
validarMovimiento()	Verifica que el jugador no atraviere paredes.
calcularPuntaje()	Calcula el puntaje final del juego.
detectarObjeto()	Determina qué objeto existe en una posición específica.
contarCeldasLibres()	Cuenta los caminos libres del mapa.
actualizarMapa()	Actualiza la posición del jugador dentro de la matriz.

juego.h

Función	Propósito	Parámetros	Retorno	Descripción
iniciarJuego()	Inicializar el juego.	Ninguno.	void	Configura el inicio del juego, carga el primer nivel e inicializa los recursos necesarios.
estadoJuego()	Actualizar la lógica del juego	Ninguno.	void	Controla movimientos, colisiones, detección de objetos, cambio de niveles y estado general del juego.
llamarDibujar(bool *status)	Dibujar elementos gráficos.	"status" estado de ejecución del juego.	void	Llama a las funciones gráficas para mostrar el mapa, jugador, estadísticas y pantallas de juego.
limpiar()	Liberar recursos	Ninguno.	void	Cierra y libera recursos gráficos, de audio y memoria utilizados por el juego
cambiarNivel()	Cambiar de nivel.	Ninguno.	void	Reinicia variables y carga el siguiente mapa del juego.
cargarMapa(const char *nombreArchivo, int mapSize)	Cargar un mapa desde un archivo.	nombreArchivo: nombre del archivo, mapSize: tamaño del mapa.	char *	Lee el contenido de un archivo de texto y lo almacena en memoria como una matriz de caracteres.
contarCaracter(char *mapa, int totalCeldas, char buscado)	Contar apariciones de un carácter.	mapa, totalCeldas, buscado.	int	Cuenta cuántas veces aparece un carácter específico dentro del mapa.
validarMovimiento(char *mapa, int mapSize, int fila, int columna)	Validar un movimiento	mapa, mapSize, fila, columna	int	Verifica si la posición destino es válida y no corresponde a una pared.
monedas, int llaves, int pasos, int niveles)	Calcular puntaje final.	monedas, llaves, pasos, niveles.	int	Obtiene la puntuación total del jugador considerando monedas, llaves, pasos y niveles completados.
detectarObjeto(char *mapa, int mapSize, int fila, int columna, char buscado)	Detectar un objeto en el mapa.	mapa, mapSize, fila, columna, buscado.	int	Cuenta cuántas veces aparece un carácter específico dentro del mapa.
contarCeldasLibres(char *mapa, int totalCeldas)	Contar caminos libres.	mapa, totalCeldas.	int	Verifica si la posición destino es válida y no corresponde a una pared.
actualizarMapa(char *mapa, int mapSize, int filaActual, int columnaActual, int filaDestino, int columnaDestino)	Actualizar posición del jugador.	mapa, mapSize, coordenadas actual y destino.	int	Modifica la matriz del mapa para reflejar el movimiento del jugador

“Bibliotecas Utilizadas”

Biblioteca	Uso
stdbool.h	Permite utilizar el tipo de dato booleano (bool), necesario para controlar estados lógicos dentro del juego.
raylib.h	Biblioteca utilizada para crear la ventana, dibujar gráficos, reproducir audio y manejar entradas del usuario.

mapas.h

Elemento	Valor	Descripción
MAP1_SIZE	60	Define el tamaño del mapa del Nivel 1. El mapa tiene 60 filas y 60 columnas.
MAP2_SIZE	62	Define el tamaño del mapa del Nivel 2. El mapa tiene 62 filas y 62 columnas.
MAP3_SIZE	64	Define el tamaño del mapa del Nivel 3. El mapa tiene 64 filas y 64 columnas.
MAP4_SIZE	70	Define el tamaño del mapa del Nivel 4. El mapa tiene 70 filas y 70 columnas.

build.sh

Sección	Propósito	Descripción
Compilación ASM	Generar el archivo objeto de NASM.	Compila todos los archivos .asm al archivo objeto procesos.o usando NASM en formato ELF64.
Compilación C	Generar los archivos objeto de C	Compila main.c, juego.c y graficos.c en archivos objeto (.o).
Enlace (Linking)	Crear el ejecutable final.	Une los archivos objeto de C y NASM para generar el ejecutable BitQuest.
Manejo de errores	Detectar fallos durante la compilación	Si ocurre un error en cualquier etapa, muestra un mensaje y finaliza el script.
Espera final	Mantener abierta la terminal.	Espera una tecla antes de cerrar la ejecución del script.

“Comandos utilizados”

Comando	Función
nasm -f elf64 *.asm -o procesos.o	Compila el archivo ensamblador y genera el objeto procesos.o.
gcc -c main.c juego.c graficos.c	Compila los archivos C sin enlazarlos todavía.
gcc main.o juego.o graficos.o procesos.o -o BitQuest ...	Enlaza todos los objetos y genera el ejecutable final.
echo	Muestra mensajes informativos al usuario.
exit 1	Finaliza el script indicando que ocurrió un error.
read aux	Espera una entrada del usuario antes de cerrar.
read -n 1 -s	Espera que se presione una tecla sin mostrarla en pantalla.

“Bibliotecas Enlazadas”

Biblioteca	Propósito
-lraylib	Biblioteca principal utilizada para gráficos, audio y entrada de usuario.
-lGL	proporciona acceso a OpenGL para renderizado gráfico
-lm	Biblioteca matemática de C.
-lpthread	Soporte para hilos de ejecución.
-ldl	Carga dinámica de bibliotecas compartidas.
-lrt	Funciones relacionadas con tiempo real y temporizadores.
-lX11	Manejo de ventanas y eventos en sistemas Linux con X Window System.

“Flujo de ejecución del script”

Paso	Acción	Resultado Esperado
1	Compilar archivo ASM	Se genera procesos.o
2	Compilar archivos C	Se generan main.o, juego.o y graficos.o.
3	Enlazar todos los objetos	Se crea el ejecutable BitQuest.
4	Mostrar mensajes de éxito o error	El usuario conoce el estado de la compilación.
5	Esperar una tecla	La terminal permanece abierta hasta que el usuario continúe.

procesos.asm

Función	Propósito	Parámetros	Retorno	Descripción
contarCaracter	Contar un carácter específico dentro del mapa.	rdi: mapa, esi: total de celdas, edx: carácter buscado.	eax: cantidad encontrada.	Recorre toda la matriz del mapa y cuenta cuántas veces aparece el carácter indicado (por ejemplo monedas o llaves).
validarMovimiento	Verificar si una posición es válida para moverse.	rdi: mapa, esi: tamaño del mapa, edx: fila destino, ecx: columna destino.	eax: 1 si puede moverse, 0 si hay una pared.	Calcula la posición solicitada y comprueba si contiene el carácter #.
calcularPuntaje	Calcular el puntaje final del jugador.	edi: monedas, esi: llaves, edx: pasos, ecx: niveles completados.	eax: puntaje total.	Aplica una fórmula que recompensa monedas, llaves y niveles completados, penalizando la cantidad de pasos realizados.
detectarObjeto	Detectar un objeto específico en una celda.	rdi: mapa, esi: tamaño del mapa, edx: fila, ecx: columna, r8: carácter buscado.	eax: 1 si coincide, 0 si no coincide.	Permite identificar monedas, llaves, puertas, salidas o cualquier otro objeto dentro del mapa.
contarCeldasLibres	Contar caminos disponibles.	rdi: mapa, esi: total de celdas.	eax: cantidad de celdas libres.	Recorre todo el mapa contando las posiciones que contienen el carácter ..
actualizarMapa	Actualizar la posición del jugador.	rdi: mapa, esi: tamaño del mapa, edx: fila actual, ecx: columna actual, r8: fila destino, r9: columna destino.	No devuelve valor (void).	Mueve al jugador a una nueva posición y reemplaza la posición anterior por un piso (.).

“Registros Utilizados”

Registro	Uso principal
RDI	Dirección base del mapa en memoria.
RSI	Tamaño del mapa o total de celdas.
RDX	Fila o carácter buscado según la función.
RCX	Columna de la posición a evaluar.
R8	Parámetro adicional (carácter buscado o fila destino).
R9	Columna destino en la actualización del mapa.
RAX/EAX	Registro utilizado para cálculos y valor de retorno.
R10	Registro auxiliar para almacenar caracteres o posiciones.
R11	Registro auxiliar para recorrer el mapa.

“Operaciones realizadas”

Función	Operación Principal
contarCaracter	Recorrido secuencial y conteo de coincidencias.
validarMovimiento	Cálculo de índice y verificación de paredes.
calcularPuntaje	Multiplicaciones y suma/resta de valores.
detectarObjeto	Comparación de caracteres en una posición específica.
contarCeldasLibres	Recorrido completo y conteo de pisos (.).
actualizarMapa	Copia y modificación de caracteres dentro de la matriz.

“Objetos del Juego Manipulados por el ASM”

Símbolo	Significado	Uso en las Funciones
P	Jugador	Se mueve mediante actualizarMapa.
.	Paso libre	Es contado por contarCeldasLibres.
#	Pared	Es detectado por validarMovimiento.
M	Moneda	Es contado por contarCaracter y detectado por detectarObjeto.
K	Llave	Es contado por contarCaracter y detectado por detectarObjeto.
D	Puerta	Es detectada por detectarObjeto.
E	Salida	Es detectada por detectarObjeto.

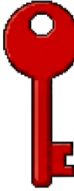
graficos.c

Función	Propósito	Parámetros	Retorno	Descripción
iniciarGraficos()	Inicializar la ventana, audio y texturas.	Ninguno.	void	Configura la ventana del juego, carga la música de fondo y las texturas de todos los elementos gráficos.
dibujar_mapa(char *mapActual, int camX, int camY, int mapSize)	Dibujar el mapa visible.	mapActual, camX, camY, mapSize.	void	Recorre la sección visible del mapa (20×20) y dibuja las texturas correspondientes a cada objeto.
dibujarJugador(int jugadorX, int jugadorY, int camX, int camY)	Dibujar al jugador.	jugadorX, jugadorY, camX, camY.	void	Dibuja la textura del personaje en la posición correspondiente dentro de la ventana visible.
actualizarMusica()	Actualizar la reproducción de música.	Ninguno.	void	Mantiene la reproducción continua de la música de fondo durante la ejecución del juego.
finGraficos()	Liberar recursos gráficos y de audio.	Ninguno.	void	Descarga las texturas, libera la música y cierra la ventana y dispositivos de audio.

“Recursos gráficos”

Variable	Tipo	Descripción
Pared	Texture2D	Textura utilizada para representar paredes (#).
Camino	Texture2D	Textura utilizada para representar caminos libres (.).
Puerta	Texture2D	Textura utilizada para representar puertas (D).
Moneda	Texture2D	Textura utilizada para representar monedas (M).
Jugador	Texture2D	Textura utilizada para representar al personaje (P).
Salida	Texture2D	Textura utilizada para representar la salida (E).
Llave	Texture2D	Textura utilizada para representar llaves (K).
musicaFondo	Music	Música de fondo reproducida durante el juego.

“Objetos dibujados en le mapa”

Símbolo	Elemento	Textura
#	Pared	Pared.png 
.	Camino	Camino.png 
M	Moneda	Moneda.png 
K	Llave	Llave.png 
D	Puerta	Puerta.png 
E	Salida	Salida.png 
P	Jugador	Jugador.png 

“Funciones de Raylib Utilizadas”

Función de Raylib	Propósito
InitWindow()	Crear la ventana del juego.
SetTargetFPS()	Definir la velocidad de actualización.
InitAudioDevice()	Inicializar el sistema de audio.
LoadMusicStream()	Cargar la música de fondo.
PlayMusicStream()	Iniciar la reproducción de música.
LoadTexture()	Cargar una textura desde un archivo.
DrawTexturePro()	Dibujar una textura escalada en pantalla.
UpdateMusicStream()	Actualizar la reproducción de música.
UnloadTexture()	Liberar una textura de memoria.
UnloadMusicStream()	Liberar la música cargada.
CloseAudioDevice()	Cerrar el sistema de audio.
CloseWindow()	Cerrar la ventana del juego.

graficos.h

Función	Propósito	Parámetro	Retorno	Descripción
iniciarGraficos()	Inicializar el entorno gráfico.	Ninguno.	void	Configura la ventana, el audio, las texturas y los recursos gráficos necesarios para ejecutar el juego.
dibujar_mapa(char *mapActual, int camX, int camY, int mapSize)	Dibujar el mapa visible.	mapActual, camX, camY, mapSize.	void	Representa gráficamente la sección visible del mapa utilizando las texturas correspondientes.
dibujarJugador(int jugadorX, int jugadorY, int camX, int camY)	Dibujar al jugador.	jugadorX, jugadorY, camX, camY.	void	Muestra al personaje en la posición correcta dentro de la ventana del juego.
actualizarMusica()	Actualizar la música de fondo.	Ninguno.	void	Mantiene la reproducción continua de la música durante la ejecución del juego.
finGraficos()	Liberar recursos gráficos	Ninguno.	void	Descarga texturas, detiene la música y cierra la ventana al finalizar el programa.

Captura de los tres niveles funcionando.

En las siguientes figuras 1, 1.2, 1.3, 1.4,1.5 se muestra el funcionamiento de el juego con sus respectivos niveles siendo el nivel 4 el ultimo y como se pidió,el de la dificultad mas alta.

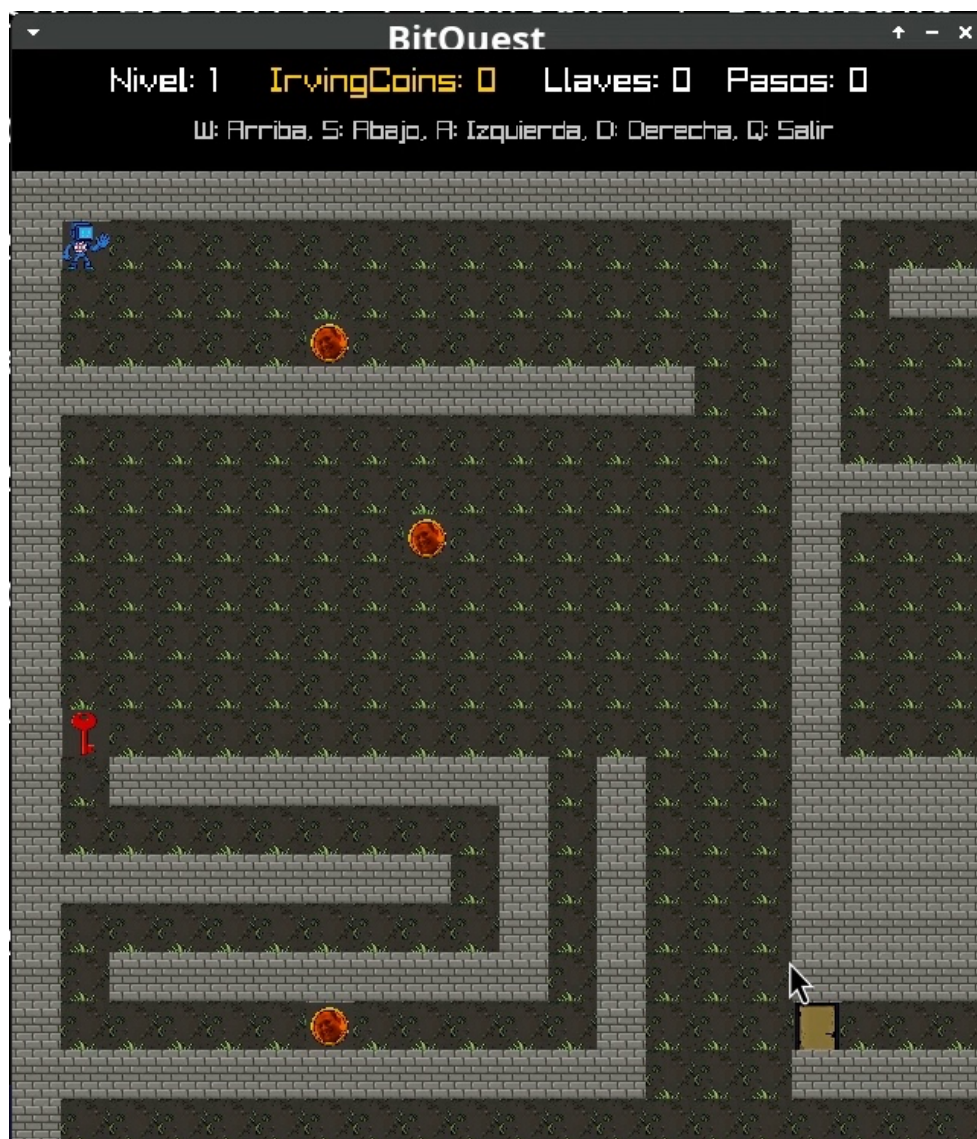


Figura 1. Nivel 1 Inicio

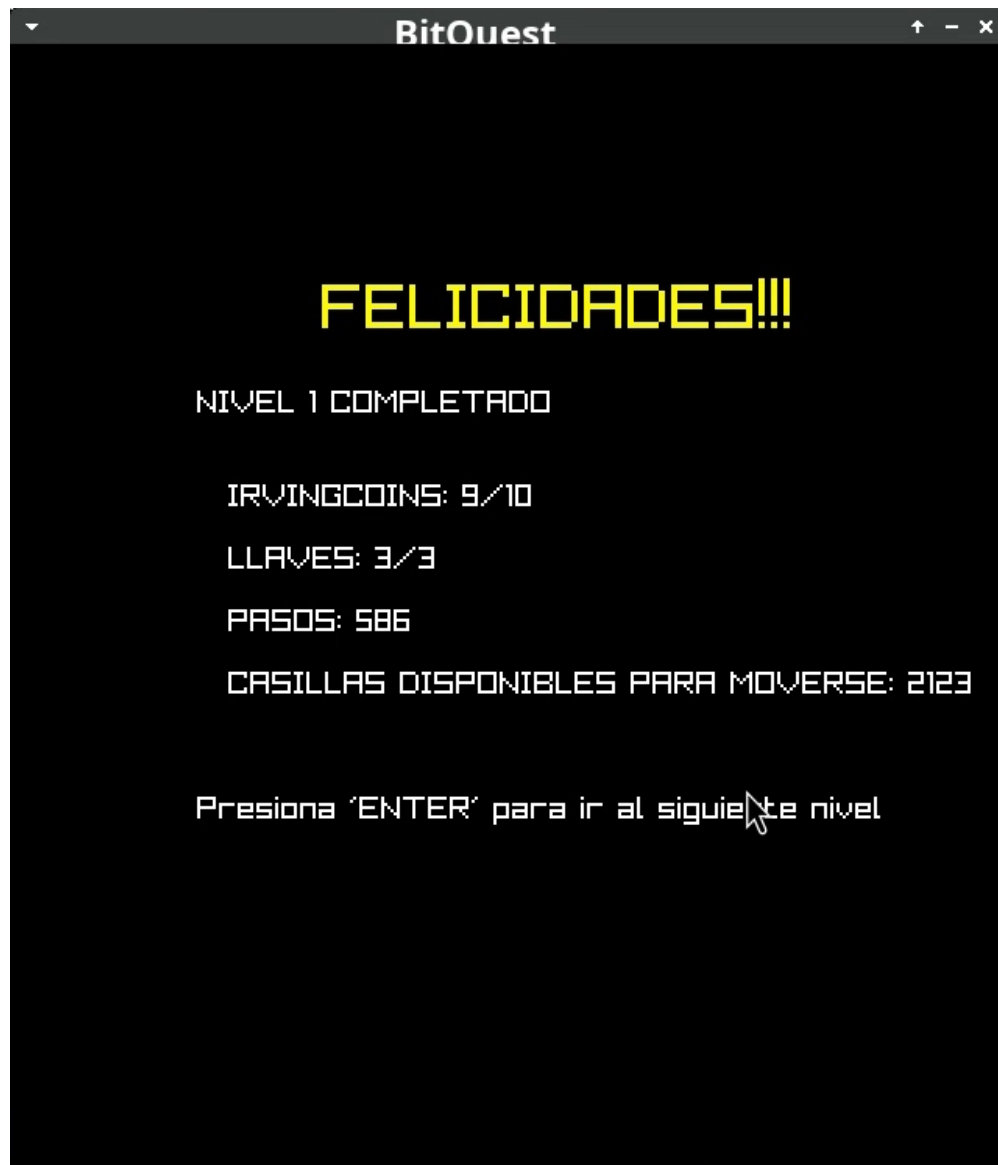


Figura 1.2. Nivel 1

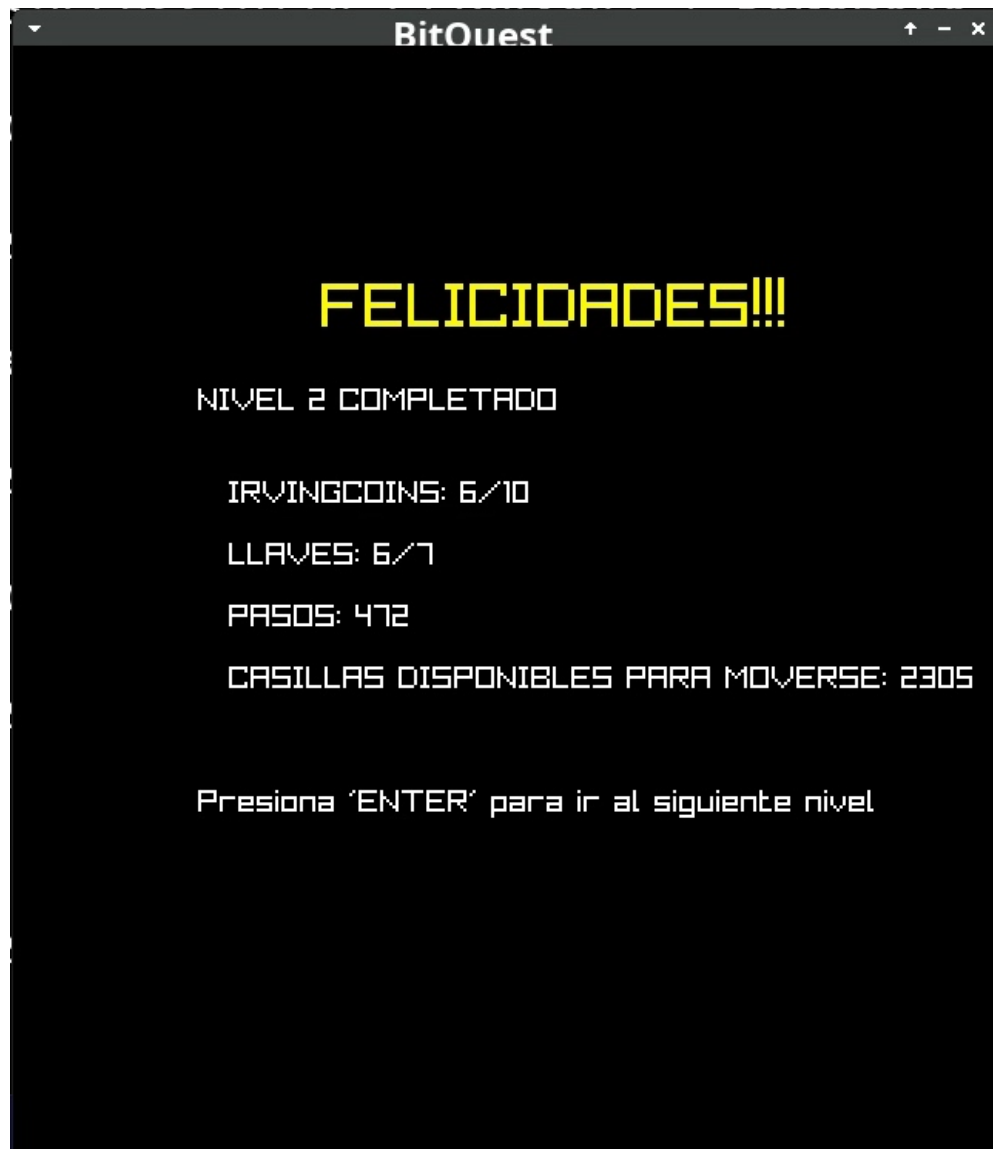


Figura 1.3. Nivel 2

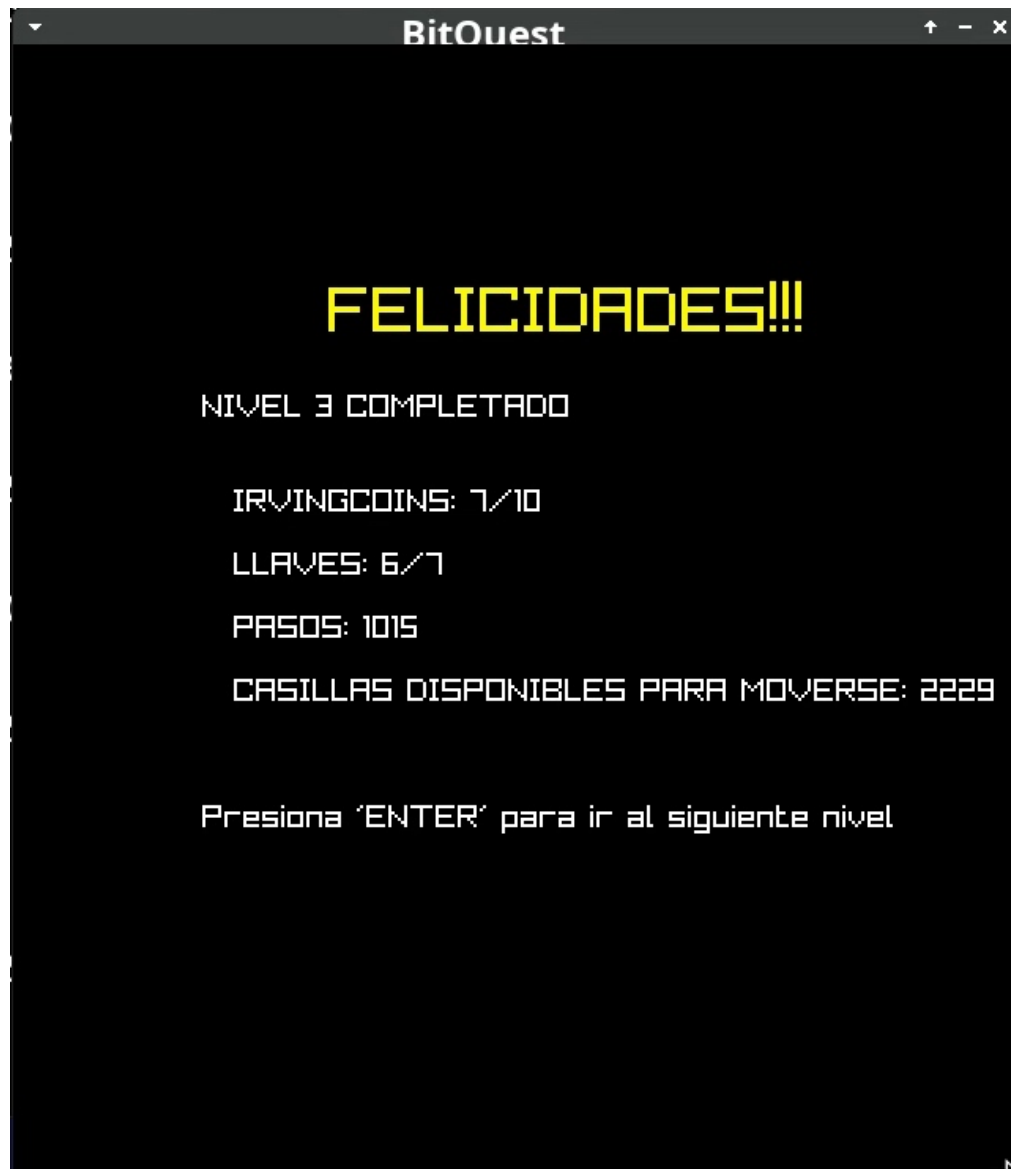


Figura 1.4. Nivel 2

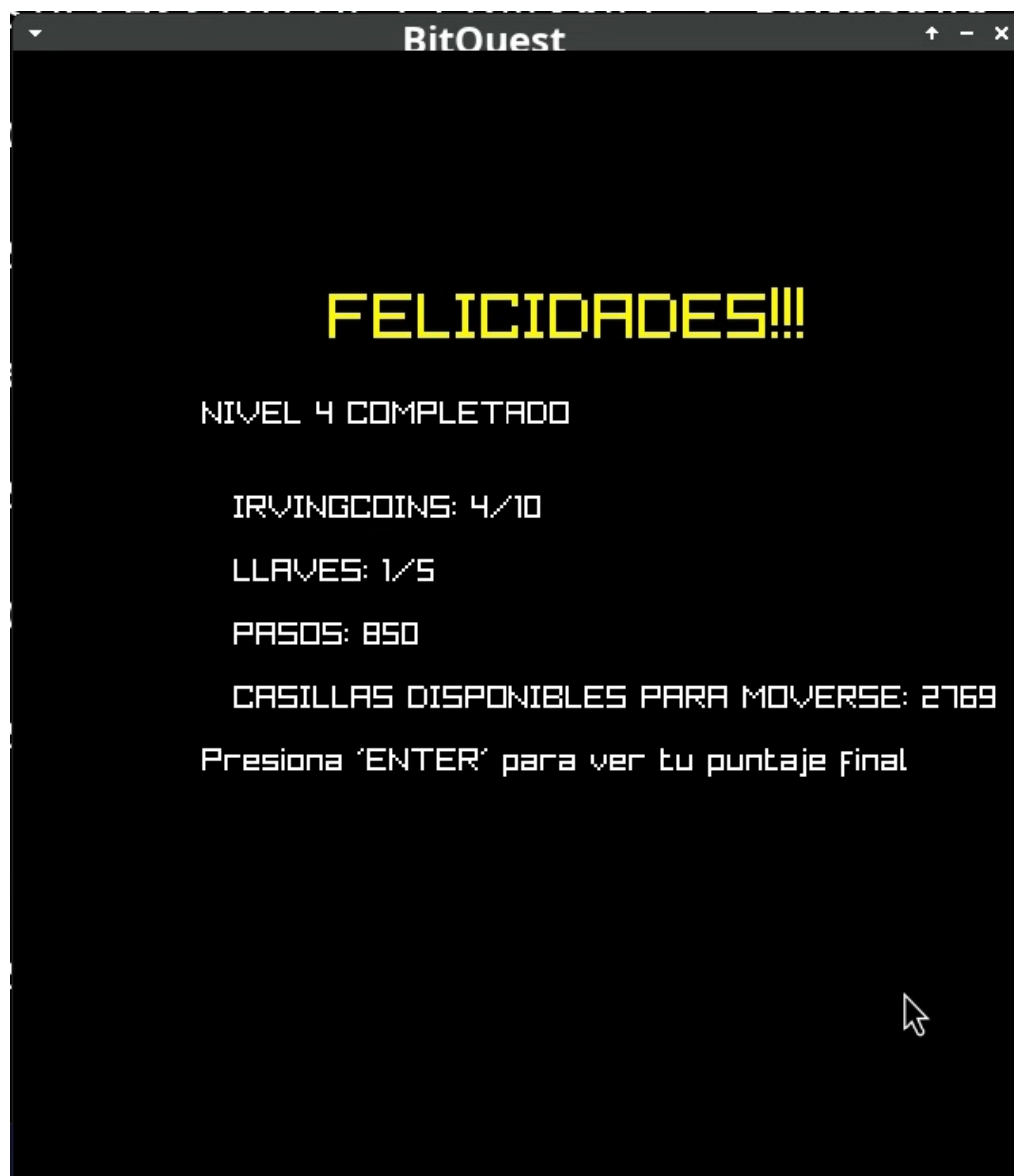


Figura 1.5. Nivel 2

Captura del resumen final del juego.

En las siguientes figuras 2 y 2.1 se muestra el resumen final del juego.

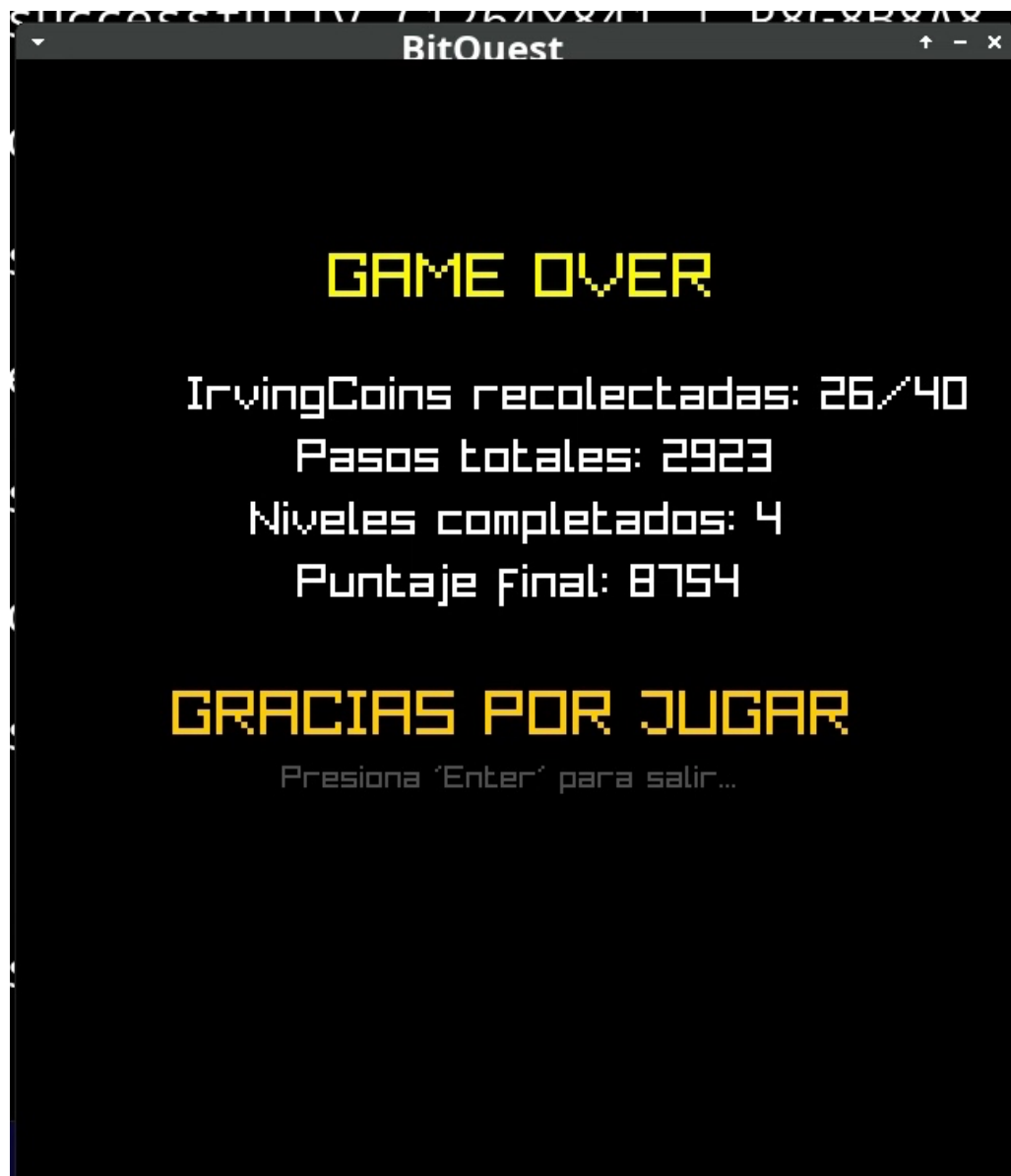


Figura 2. Game Over

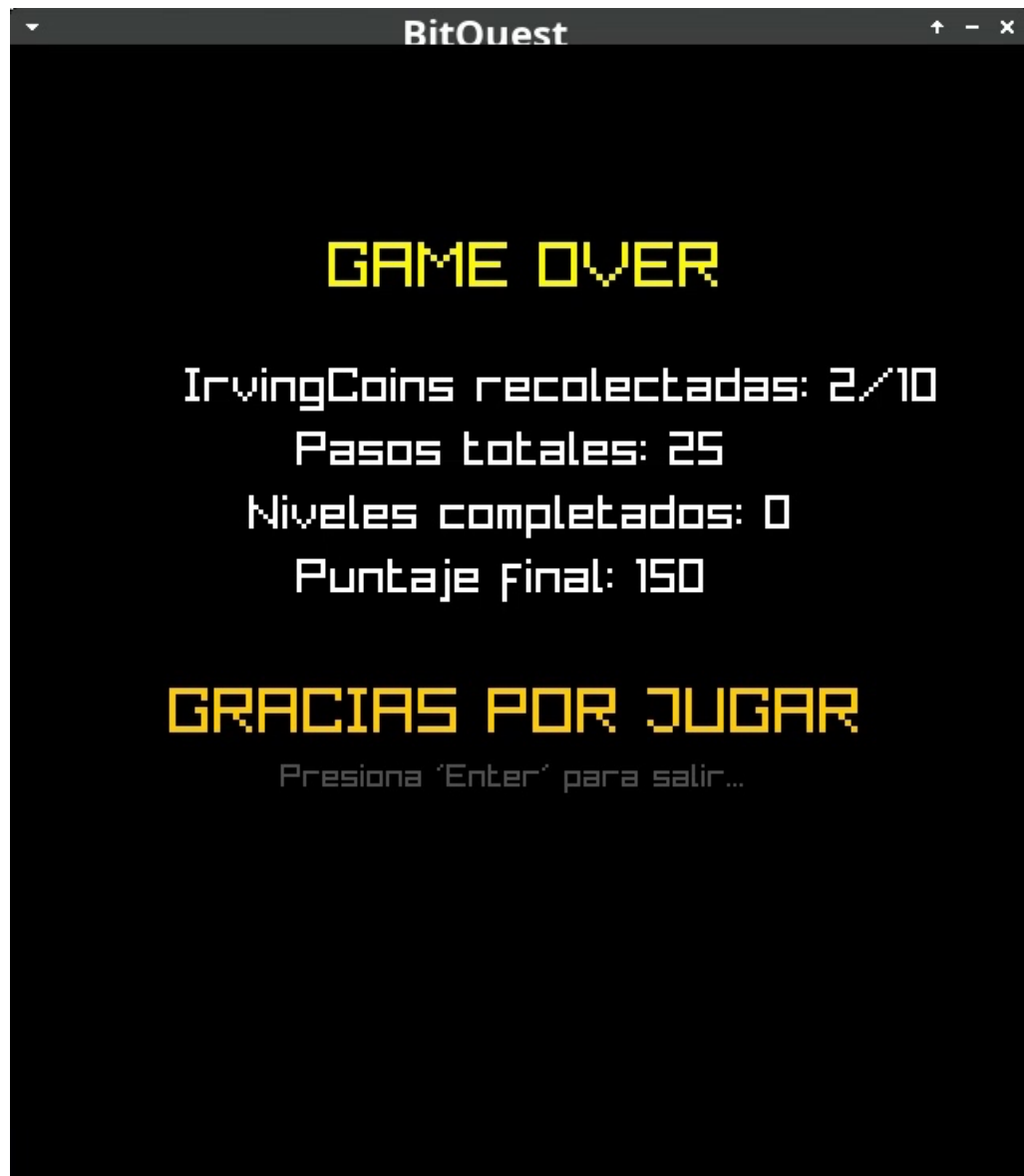


Figura 2.1. Puntaje final del juego

Instrucciones de compilación y ejecución.

Este programa fue escrito en un sistema operativo Linux Mint 22.3, por lo que para las instrucciones de compilación se usaron las establecidas para linux.

Se creó un script "build.sh" que ejecuta las instrucciones de compilación de NASM:

```
nasm -f elf64 *.asm -o "procesos".o
```

De compilación de los archivos de C:

```
gcc -c main.c juego.c graficos.c
```

Y por último el ensamblado de los archivos:

```
gcc main.o juego.o graficos.o procesos.o -o "BitQuest" -lraylib -lGL -lm -lpthread -ldl -lrt -lX11
```

Por lo que para realizar la compilación del programa solo hay que ejecutar dicho script desde la terminal:

```
./build.sh
```

Y el programa ya ensamblado se guarda con el nombre "BitQuest", y de igual manera solo hay que ejecutarlo desde la terminal:

```
./BitQuest
```

Conclusiones.

El desarrollo de **BitQuest** permitió integrar conocimientos de programación en **C**, programación en **ensamblador x86-64**, manejo de archivos y desarrollo gráfico utilizando la biblioteca Raylib. A lo largo del proyecto se implementó un videojuego funcional basado en niveles, donde el jugador debe explorar mapas, recolectar monedas y llaves, abrir puertas y encontrar la salida para avanzar.

Uno de los aspectos más importantes fue la integración de distintos lenguajes de programación. Mientras que C se utilizó para la lógica principal del juego y la interacción con el usuario, el ensamblador se empleó para realizar operaciones específicas sobre los mapas, como la validación de movimientos, el conteo de objetos y el cálculo del puntaje final. Esto permitió comprender cómo diferentes tecnologías pueden complementarse dentro de un mismo proyecto.

Además, se implementó un sistema de niveles cargados desde archivos externos, facilitando la organización y la posibilidad de agregar nuevos escenarios sin modificar el código principal. También se desarrolló una interfaz gráfica con texturas, música de fondo y paneles informativos que mejoran la experiencia del usuario.

En conclusión, **BitQuest** es un proyecto que demuestra la aplicación práctica de los conocimientos adquiridos durante el curso, combinando programación de alto y bajo nivel para desarrollar una aplicación funcional, organizada y capaz de integrar diferentes tecnologías en un solo sistema.

Referencias Bibliográficas.

Sv, V. (2025). *BitQuest: Proyecto Final de Lenguaje Ensamblador*. GitHub. Recuperado de <https://github.com/suarez-vv/BitQuest-Game>

Free Software Foundation. (s.f.). *GNU Compiler Collection (GCC) Documentation*. Recuperado de <https://gcc.gnu.org/onlinedocs/>

Free Software Foundation. (s.f.). *GNU Bash Reference Manual*. Recuperado de <https://www.gnu.org/software/bash/manual/>

Intel Corporation. (s.f.). *Intel® 64 and IA-32 Architectures Software Developer's Manual*. Recuperado de <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

Netwide Assembler Project. (s.f.). *NASM Documentation*. Recuperado de <https://www.nasm.us/docs.php>

Raylib Technologies. (s.f.). *Raylib Documentation*. Recuperado de <https://www.raylib.com/index.html>